

Orchestrer l'intelligence

*Le guide complet pour débutants absolus —
comprendre comment plusieurs IA travaillent ensemble pour construire un vrai logiciel*

Compagnon du talk • Montréal • 18 juin 2026
Faïçal Sawadogo • Moussa Balla

Sommaire

Avant-propos

Chapitre 0 — Cinq mots à comprendre avant de commencer

Chapitre 1 — La grande métaphore : une entreprise miniature

Chapitre 2 — Vibe coding vs vibe engineering

Chapitre 3 — Agent généraliste vs spécialiste

Chapitre 4 — Comment Claude lit ton projet

Chapitre 5 — Anatomie d'un skill : décorticage ligne par ligne

Chapitre 6 — Cycle de vie d'un agent forké

Chapitre 7 — Skill atomique vs orchestrateur

Chapitre 8 — Décorticage d'un workflow réel

Chapitre 9 — Le piège du mega-agent

Chapitre 10 — Comment l'orchestrateur décide de l'ordre et quand rappeler un skill

Chapitre 11 — Où vit la mémoire

Chapitre 12 — Les gates en détail

Chapitre 13 — Les 8 règles d'or pour orchestrer un workflow multi-agents

Chapitre 14 — Démo ServiceCat décodée

Chapitre 15 — Démarrer lundi matin

Glossaire enrichi

Annexe — Pour aller plus loin

Orchestrer l'intelligence — Le guide complet pour débutants absolus

Un tutoriel pour comprendre comment plusieurs IA travaillent ensemble pour construire un vrai logiciel

Ancré sur ServiceCat, le projet de démonstration du talk de Moussa Balla — Montréal, 18 juin 2026

Avant-propos

Bonjour. Si tu lis ces lignes, c'est probablement que tu as entendu parler d'IA, d'agents, ou de Claude Code, et que tu te demandes ce qui se passe vraiment derrière les mots.

Ce guide est écrit pour TOI. Pas pour un développeur. Pas pour un expert. Pour quelqu'un qui n'a JAMAIS touché à l'intelligence artificielle de façon sérieuse.

À la fin de ce guide, tu vas comprendre :

- Ce qu'est un *agent* IA (un assistant logiciel qui agit, pas qui répond).
- Pourquoi on en utilise plusieurs en même temps au lieu d'un seul gros.
- Comment ils se parlent, se passent le travail, et savent quand s'arrêter.
- Comment fonctionne *ServiceCat*, le projet que Moussa va montrer en démo le 18 juin 2026 à Montréal.

Le ton sera celui d'un grand frère ou d'une grande sœur qui t'explique le fonctionnement d'un atelier de jouets. Phrases courtes. Beaucoup d'exemples. Aucun jargon non expliqué.

Tu peux le lire dans l'ordre ou sauter d'un chapitre à l'autre. Mais si c'est la première fois, lis-le dans l'ordre. Chaque chapitre prépare le suivant.

Prêt ? On commence.

Chapitre 0 — Cinq mots à comprendre avant de commencer

Avant tout, posons cinq mots de base. Ces cinq mots vont revenir partout. Si tu les comprends, tu comprends 80% du reste.

1. IA (Intelligence Artificielle)

Une *IA* est un programme informatique qui imite certains comportements humains : lire, écrire, raisonner, traduire, coder. Ce n'est pas une personne. C'est un logiciel très avancé qui a appris à partir de milliards de textes.

Exemple : Claude est une IA. ChatGPT est une IA. Ce sont des programmes, pas des cerveaux.

2. Modèle

Un *modèle* est la “version” précise d'une IA. C'est comme une voiture : il existe la marque (Claude) et le modèle (Opus 4.7). Le modèle décide la taille du cerveau, la vitesse, et la quantité d'information qu'il peut traiter à la fois.

Exemple : Claude Opus 4.7 1M est un modèle. Le “1M” veut dire qu'il peut lire jusqu'à un million de *tokens* à la fois (on définit *token* juste en dessous).

3. Prompt

Un *prompt* est le message que tu envoies à l'IA. C'est ta question, ta demande, ton instruction. C'est exactement comme parler à un assistant : tu dis ce que tu veux, il fait.

Exemple : “Écris-moi une fonction qui calcule la moyenne” est un prompt.

4. Token

Un *token* est l'unité de mesure de texte que l'IA voit. Pour faire simple : un token, c'est environ trois quarts de mot français. La phrase “Bonjour, comment vas-tu ?” fait à peu près 6 tokens.

Pourquoi c'est important ? Parce que les IA ont une limite : elles ne peuvent traiter qu'un certain nombre de tokens à la fois. C'est leur fenêtre de vision.

Exemple : Claude Opus 4.7 1M peut traiter 1 million de tokens. Soit environ 750 000 mots français. Soit l'équivalent de 5 ou 6 livres entiers.

5. Contexte

Le *contexte* est tout ce que l'IA “voit” en même temps quand elle te répond : ta question, l'historique de la conversation, les fichiers qu'on lui a montrés, les instructions qu'on lui a données.

C'est comme la mémoire de travail d'un humain. Au-delà d'une certaine quantité, l'IA “oublie” ce qui est trop loin en arrière.

Exemple : Quand tu parles à Claude et que tu lui montres un fichier, ce fichier entre dans son contexte. Si tu lui donnes 10 fichiers très gros, son contexte se remplit, et il devient plus lent ou confus.

À retenir : une IA = un programme. Un modèle = sa version. Un prompt = ta question. Un token = un petit bout de texte. Un contexte = sa mémoire de travail immédiate.

Avec ces cinq mots, on peut maintenant commencer pour de vrai.

Chapitre 1 — La grande métaphore : une entreprise miniature

Imagine un atelier de jouets en bois. Pas une usine. Un atelier de quartier.

Dans cet atelier, il y a :

- **Un chef d'atelier.** C'est lui qui reçoit les commandes des clients. Il ne fabrique rien lui-même. Il décide qui fait quoi, dans quel ordre, et il vérifie que tout est fini avant de livrer.
- **Des artisans spécialisés.** L'un coupe le bois. L'un ponce. L'un peint. L'un assemble. L'un emballe. Chacun connaît une seule chose, mais il la fait très bien.
- **Un livre de règles de la maison.** Accroché au mur. Il dit “ici on ne travaille jamais sans gants”, “ici on signe chaque jouet”, “ici on jette les chutes dans la bonne poubelle”. Tout le monde connaît ce livre.
- **Des points de contrôle.** Avant de passer un jouet de l'atelier de peinture à l'atelier d'emballage, quelqu'un vérifie qu'il est bien sec, qu'il n'y a pas de traces. Si oui, on passe. Si non, retour à la peinture.

Maintenant, traduisons cet atelier en termes d'IA.

Atelier de jouets	Monde de Claude Code
Le chef d'atelier	L'orchestrateur (l'agent principal qui coordonne)
Les artisans spécialisés	Les <i>skills</i> (compétences spécifiques)
Le livre de règles de la maison	Le fichier <code>CLAUDE.md</code> (les règles du projet)
Les sous-traitants ponctuels	Les <i>sous-agents</i> (instances IA temporaires)
Les points de contrôle	Les <i>gates</i> (barrières de qualité)

Voici un schéma simplifié de ce qui se passe quand tu demandes “ajoute une fonctionnalité à mon site” :



À retenir : Une équipe d’IA, c’est exactement comme un atelier humain. Un chef qui coordonne, des spécialistes qui exécutent, des règles communes, et des contrôles de qualité entre les étapes.

Le reste du guide va te détailler chaque morceau.

Chapitre 2 — Vibe coding vs vibe engineering

Avant d’aller plus loin, il faut comprendre l’évolution des cinq dernières années. C’est ce que Moussa montre dans les slides 3 à 7 de son talk.

L’évolution en quatre temps

Temps 1 — L’autocomplete (2020-2022). L’IA propose la fin de ta phrase pendant que tu écris du code. Comme l’autocomplete du téléphone. Tu restes le pilote, l’IA est juste un soutien clavier.

Temps 2 — La conversation (2022-2023). L’IA répond à des questions dans un *chat*. “Comment je fais ça ?” “Voilà le code.” Tu copies, tu colles, tu adaptes. L’IA ne touche pas à tes fichiers.

Temps 3 — L’agent (2023-2025). L’IA peut LIRE tes fichiers, les MODIFIER, LANCER des commandes. Elle ne se contente plus de proposer du texte : elle AGIT. C’est un grand saut. On dit qu’elle est *agentique*.

Temps 4 — L’orchestration (2025+). Plusieurs agents IA travaillent ensemble. Un agent dirige, d’autres exécutent. Chacun a un rôle précis. C’est ce qu’on appelle l’*orchestration multi-agents*. C’est le sujet de ce guide.

Vibe coding vs vibe engineering

Moussa utilise deux expressions importantes dans son talk.

Le *vibe coding* (mot anglais qui signifie littéralement “coder à l’instinct”) : tu demandes une fonctionnalité à une IA, tu prends ce qu’elle produit, tu testes, tu pries pour que ça marche. C’est rapide. C’est fragile. C’est imprévisible.

Le *vibe engineering* (ingénierie à l’instinct, mais avec un cadre) : tu construis une vraie équipe d’IA avec des rôles, des règles, des points de contrôle. Tu pilotes plusieurs agents. Tu gardes les bonnes pratiques d’ingénierie : tests, revues, traçabilité.

Vibe coding	Vibe engineering
Un seul agent, fait tout	Plusieurs agents, chacun son rôle
Pas de plan formel	Plan obligatoire avant code
Pas de critère d'acceptation	6 critères à valider (AC-1 à AC-6)
Pas de revue automatique	Revue + audit sécurité systématiques
Tu pries que ça marche	Tu sais que ça marche
Rapide à démarrer, lent à corriger	Plus structuré, beaucoup plus fiable

Same models, different system

La slide 7 du talk donne une phrase clé : *same models, different system* (les mêmes modèles, mais un système différent).

Cela veut dire quoi ? Que la magie n'est PAS dans le modèle d'IA. Ce n'est pas parce que tu utilises Claude Opus 4.7 plutôt que Claude Sonnet 4 que tu vas réussir. La magie est dans le SYSTÈME que tu construis autour du modèle : les skills, les règles, les gates, l'orchestration.

Tu peux faire du vibe coding avec Opus 4.7 (très bon modèle) et obtenir un résultat médiocre. Tu peux faire du vibe engineering avec Sonnet 4 (modèle plus modeste) et obtenir un résultat solide. Le système gagne sur le modèle.

À retenir : *Le vibe engineering, c'est encadrer l'IA pour la rendre fiable. Ce n'est pas le modèle qui décide. C'est l'organisation autour du modèle.*

Chapitre 3 — Agent généraliste vs spécialiste

Imagine que tu vas à l'hôpital pour une douleur au genou.

Cas 1 : Tu vois UN médecin généraliste. Il connaît un peu tout : le cœur, les poumons, la peau, les os. Il fait son possible. Il pose des questions, il tâte, il propose. Mais il n'a pas fait dix ans d'études sur les genoux. Il pourrait passer à côté de quelque chose.

Cas 2 : Tu vois D'ABORD un généraliste pour le diagnostic global. Puis il t'envoie chez un orthopédiste (spécialiste des os). L'orthopédiste t'envoie chez un radiologue (spécialiste des

images). Le radiologue te renvoie à l'orthopédiste avec les résultats. L'orthopédiste te renvoie chez un kinésithérapeute (spécialiste de la rééducation).

Dans le cas 2, chaque professionnel sait UNE chose, mais il la sait à fond. Et un coordinateur (souvent le généraliste) organise leur enchaînement.

C'est exactement la différence entre un *agent généraliste* et une équipe d'*agents spécialistes*.

Pourquoi des spécialistes ?

Un agent généraliste qui essaie de tout faire :

- Doit garder en tête toutes les règles de tous les domaines en même temps.
- A un contexte qui se remplit vite (tous les fichiers qu'il regarde restent en mémoire).
- Mélange les phases : il commence à coder avant d'avoir fini de planifier.
- Donne souvent des résultats moyens : pas de spécialisation profonde.

Un agent spécialiste :

- Connaît UNE chose à fond.
- Démarre avec un contexte propre pour chaque mission.
- Ne se laisse pas distraire.
- Donne des résultats nets sur sa spécialité.

Exemple chiffré

Sur ServiceCat, ajouter une fonctionnalité moyenne (la *feature* F-12, qui ajoute le versionnement des scorecards) coûte :

- Avec UN gros agent qui fait tout : environ **960 000 tokens**.
- Avec UNE équipe de spécialistes orchestrés : environ **120 000 tokens**.

C'est un facteur **8 fois moins**. Tu dépenses 8 fois moins d'argent en appels d'API. Le résultat est plus fiable. On verra pourquoi en détail au chapitre 9.

À retenir : *Un seul agent qui fait tout = un médecin qui pratique toutes les spécialités. Plusieurs agents spécialistes coordonnés = un vrai hôpital. Le deuxième modèle gagne presque toujours.*

Chapitre 4 — Comment Claude lit ton projet

C'est un chapitre essentiel. Trop souvent, les débutants ne comprennent pas COMMENT Claude prend connaissance d'un projet. Une fois que tu comprends ce mécanisme, tu peux organiser ton projet intelligemment.

Deux types de fichiers

Dans le monde de Claude Code, il existe deux familles de fichiers :

Famille 1 — Les fichiers à nom RÉSERVÉ. Leur nom est imposé par le système. Si tu changes leur nom, Claude ne les trouve plus. Le système les cherche par leur nom exact.

- `CLAUDE.md` (les règles globales du projet)
- `settings.json` (les réglages techniques de la session)
- `SKILL.md` (le contenu de chaque skill, dans son dossier)
- `.mcp.json` (les outils externes branchés)
- `.claudeignore` (la liste des fichiers à ignorer)

Famille 2 — Les fichiers à NOM LIBRE. Tu peux les appeler comme tu veux. Tant qu'on y fait référence correctement, tout fonctionne.

- `WORKFLOW.md` (peut s'appeler `toto.md`, `README.md`, `process.md`, peu importe)
- La documentation dans `docs/`
- Les guides d'onboarding
- Les notes d'équipe

Exemple concret : Sur ServiceCat, il y a un fichier `WORKFLOW.md` qui décrit comment l'équipe utilise Claude Code. Si demain on le renomme `EQUIPE.md`, il faut juste mettre à jour les liens dans `CLAUDE.md` qui pointaient vers lui. Aucun problème technique.

Deux types de chargement

C'est l'autre point critique à comprendre. Tous les fichiers ne sont pas lus de la même façon.

Type 1 — Chargement AUTOMATIQUE au démarrage de la session. Quand tu lances Claude Code, certains fichiers sont lus immédiatement. Leur contenu entre dans le contexte de la session. Claude les a "en tête" dès la première question.

- `CLAUDE.md` (tout son contenu)

- `settings.json` (tous les réglages)
- Les *frontmatters* (en-têtes) des `SKILL.md` (uniquement les 3-5 premières lignes)
- `.mcp.json` (configuration des outils)
- `.claudeignore` (liste d'exclusions)

Type 2 — Lecture À LA DEMANDE. Ces fichiers ne sont JAMAIS lus au démarrage. Ils sont lus seulement quand quelqu'un (toi ou un agent) en a besoin.

- `WORKFLOW.md` (lu si tu poses une question dessus)
- `README.md` (lu si pertinent)
- Le CORPS des `SKILL.md` (lu seulement quand le skill est invoqué)
- Tous les fichiers de code (`.py` , `.tsx` , etc.)
- Les fichiers de documentation dans `docs/`

Pourquoi cette distinction est importante ? *Parce que le contexte est limité. Si Claude chargeait TOUT au démarrage, sa fenêtre serait pleine avant même que tu poses ta première question.*

Tableau récapitulatif

Fichier	Nom imposé ?	Lu automatiquement ?	Quand est-il lu ?
CLAUDE.md	Oui	Oui	Au démarrage
settings.json	Oui	Oui	Au démarrage
SKILL.md (frontmatter)	Oui	Oui	Au démarrage (en-tête seulement)
SKILL.md (corps)	Oui	Non	Quand le skill est invoqué
.mcp.json	Oui	Oui	Au démarrage
.claudeignore	Oui	Oui	Au démarrage
WORKFLOW.md	Non	Non	À la demande
README.md	Non (mais utile)	Non	À la demande
Fichiers de code	Non	Non	À la demande
Documentation docs/	Non	Non	À la demande

Pourquoi c'est génial

Cette séparation permet une chose précieuse : l'équipe peut avoir BEAUCOUP de skills, BEAUCOUP de documentation, mais Claude ne paye au démarrage que pour les en-têtes. Il sait que les compétences existent (il a lu leur frontmatter) mais il ne charge le détail que quand il en a besoin.

C'est comme une bibliothèque : tu as 10 000 livres sur les étagères, mais tu ne lis que ceux que tu prends. Le catalogue (frontmatter) te dit ce qui existe. Le contenu (corps) reste sur l'étagère jusqu'à ce que tu le sortes.

À retenir : Les fichiers à nom réservé sont les piliers. Ils sont auto-chargés. Tout le reste est lu à la demande. Cette séparation permet à Claude de connaître l'inventaire sans lire toute la bibliothèque.

Chapitre 5 — Anatomie d'un skill : décorticage ligne par ligne

Un *skill* (compétence) est l'unité de base d'un système Claude Code. C'est un dossier qui contient un fichier `SKILL.md`. Ce fichier décrit ce que le skill sait faire et comment il le fait.

Sur ServiceCat, il existe 18 skills à ce jour. Tu vas tous les voir dans la liste plus bas. Mais d'abord, on va en décortiquer UN, ligne par ligne, pour que tu comprennes la structure.

Le skill choisi : `audit-security`. C'est celui qui vérifie qu'on n'a pas oublié de protections de sécurité avant d'envoyer du code en production. Son fichier complet est dans `/Users/moussab/Github/genai-demo/.claude/skills/audit-security/SKILL.md`.

Le frontmatter

Les 5 premières lignes du fichier ressemblent à ceci :

```
---
name: audit-security
description: Scan code for security violations S1-S8 (auth, tenant isolation, RBAC, rat
allowed-tools: Read, Grep, Glob
---
```

C'est le *frontmatter* (en-tête de fichier en anglais). C'est la carte d'identité du skill. Comme l'étiquette d'un médicament.

Décortiquons ligne par ligne.

name: audit-security

Le nom de la compétence. C'est aussi le nom que tu tapes pour l'invoquer (avec un slash devant) : `/audit-security`.

description: Scan code for security violations S1-S8 ...

La description courte. C'est la phrase la plus importante du fichier. Pourquoi ? Parce que c'est ce que l'orchestrateur lit pour décider SI ce skill est utile pour la tâche en cours.

Imagine le chef d'atelier qui regarde l'étagère des artisans. Chaque artisan a une petite pancarte au-dessus de sa tête : "moi je peins", "moi je ponce", "moi j'emballe". Le chef lit les pancartes et choisit qui appeler. La description, c'est la pancarte.

allowed-tools: Read, Grep, Glob

La liste des outils que l'artisan a le droit d'utiliser. Ici, le skill a le droit de LIRE des fichiers (`Read`), de CHERCHER du texte dedans (`Grep`), et de LISTER des fichiers (`Glob`). Il n'a PAS le droit d'écrire ou de modifier.

C'est une protection : on confie un travail à un agent, et on limite ce qu'il peut faire. Comme donner un trousseau de clés à un employé : il a les clés du bureau, mais pas celles du coffre-fort.

À retenir : *Un skill = un contrat. Trois champs minimum dans son en-tête : son nom, sa description, ses outils autorisés. Ces trois lignes sont auto-chargées au démarrage de la session.*

Le corps du skill

Après le frontmatter vient le CORPS du `SKILL.md`. C'est là que sont les instructions détaillées pour l'agent. Pour `audit-security`, le corps contient :

- Une explication de la mission ("Tu es un auditeur de sécurité strict, tu signales sans réparer").
- La liste des 8 règles de sécurité à vérifier (S1 à S8).
- La procédure étape par étape.
- Le format exact du rapport à produire.
- La liste des choses interdites.

Voici un extrait de la procédure :

```
1. Determine scope. Default: changed files vs the trunk
2. For each file, scan against ALL rules.
3. Classify each finding by severity (CRITICAL/HIGH/MEDIUM/LOW)
4. Output a structured report.
```

Et un extrait de la table des règles :

S1	Authentication	Chaque route non publique a Depends(get_current_user)
S2	Tenant isolation	Chaque requête multi-tenant filtre par workspace_id
S3	RBAC	Chaque mutation a require_capability(...)
S4	Rate limiting	Chaque mutation a Depends(rate_limit(...))
S5	Audit logging	Chaque mutation a Depends(audit_action(...))
S6	No raw SQL	Pas de SQL formaté avec interpolation de chaîne
S7	Secrets hygiene	Pas de clé API en dur, pas de .env commité
S8	HTTP hygiene	Tous les appels externes ont un timeout explicite

Pourquoi c'est pédagogique : la première ligne du tableau (S1) signifie “vérifie que chaque porte de l'application demande à l'utilisateur qui il est avant de le laisser entrer”. La deuxième ligne (S2) signifie “vérifie que les données d'une équipe ne se mélangent jamais avec celles d'une autre”. Et ainsi de suite.

Le corps va aussi jusqu'à donner le format exact du rapport :

```
/audit-security report – 8 files scanned

CLEAN: 5 files
VIOLATIONS: 3 files

CRITICAL (1)
-----
S2 – Tenant isolation missing
  finding_repository.py:54
  Code: return await self.db.execute(...)
  Issue: Query does not filter by workspace_id
  Fix: Add .where(Finding.workspace_id == ws.id)
```

Le skill ne renvoie pas un texte vague. Il renvoie un rapport STRUCTURÉ, lisible, avec les fichiers, les lignes, le code en cause, et la correction suggérée.

Le prompt d'aujourd'hui est le skill de demain

Tu vois ce qui se passe ? Le `SKILL.md`, c'est juste un GROS prompt qu'on a écrit une fois pour toutes. Au lieu de réécrire “vérifie ces 8 règles, classifie par sévérité, formate ainsi...” à chaque conversation, on l'écrit UNE FOIS dans un fichier, et on l'invoque avec une seule commande : `/audit-security`.

Le skill est donc une cristallisation d'un savoir-faire. Le prompt qu'on tapait hier devient le skill de demain. Chaque skill bien écrit, c'est une heure de prompt-engineering qu'on n'aura plus à refaire.

Les 18 skills de ServiceCat

Voici la liste complète des skills actuellement présents dans `/Users/moussab/Github/genai-demo/.claude/skills/`. Chacun fait UNE chose.

Skill	Description en une phrase
notify	Envoie des messages sur Slack pour prévenir d'une nouvelle importante.
devops	Surveille la chaîne de production et dit où ça casse.
commit-sc	Sauvegarde les modifications dans Git avec un message formaté.
new-endpoint	Crée une nouvelle porte d'entrée web sécurisée avec tous les verrous.
work-issues	Robot autonome qui prend les tâches une par une et les résout.
explore-codebase	Lit le code sans le modifier pour comprendre comment il marche.
plan-feature	Dessine le plan exact d'une fonctionnalité avec les critères de succès.
audit-security	Inspecte le code pour trouver les failles de sécurité.
triage	Classe les problèmes par urgence et crée des tickets.
review-pr	Vérifie les changements d'un collègue avant publication.
implement	Pilote toutes les étapes d'une fonctionnalité du plan au test.
write-docs	Écrit la documentation pour que les autres comprennent.
frontend-design	Crée les beaux écrans et boutons que l'utilisateur voit.
audit-service	Vérifie si un service respecte tous les critères de qualité.
work-findings	Robot qui répare les problèmes trouvés par les audits.
simplify	Nettoie le code récent pour éviter les répétitions.
create-pr	Envoie le code et crée la demande de fusion documentée.
new-scorecard	Crée un nouveau type d'audit personnalisé.

Chaque ligne de ce tableau correspond à un fichier `SKILL.md` dans son propre dossier. Tous ont la même structure : frontmatter + corps détaillé.

À retenir : *Un skill est une compétence figée dans un fichier. Le frontmatter dit qui il est. Le corps dit comment il travaille. Un projet sérieux a des dizaines de skills, chacun spécialisé sur une seule chose.*

Chapitre 6 — Cycle de vie d'un agent forké

Ce chapitre est crucial. Si tu comprends ce qui suit, tu as compris le mécanisme de fond de l'orchestration multi-agents.

Qu'est-ce qu'un fork ?

Le mot *fork* (en anglais : fourchette, mais surtout “embranchement”) désigne le fait de créer une copie indépendante d'un agent IA. C'est comme cloner un employé pour une mission précise, en lui donnant une ardoise propre.

Quand tu tapes `/audit-security` dans Claude Code, il ne se passe PAS ce que la plupart des gens imaginent. Il ne se passe pas “l'agent principal lit le skill et l'exécute”. Il se passe une chose plus subtile et plus puissante.

Les 5 étapes du cycle de vie

Voici ce qui arrive, étape par étape, quand un skill est invoqué.

Étape 1 — Reconnaissance de la commande. L'agent principal (l'orchestrateur, ou TOI si tu tapes la commande directement) voit `/audit-security`. Il comprend que c'est une invocation de skill.

Étape 2 — Lecture du frontmatter (déjà fait au démarrage). Le frontmatter a déjà été chargé au démarrage de la session. L'orchestrateur sait donc déjà que ce skill existe, ce qu'il fait, et quels outils il peut utiliser.

Étape 3 — Fork d'une sous-session avec contexte VIDE. C'est l'étape magique. Le système crée un nouvel agent IA. Cet agent démarre avec : - Le corps du `SKILL.md` (qu'il lit pour la première fois). - Les arguments éventuels que tu as passés (par exemple `/audit-security src/`). - Un contexte de session VIDE par ailleurs.

L'agent forké ne sait rien de tes conversations précédentes. Il ne sait pas qui est l'humain. Il ne sait pas ce que tu as fait avant. Il a une mission, des instructions, des outils. C'est tout.

Étape 4 — Exécution. L'agent forké exécute sa mission. Il lit des fichiers, fait des recherches, raisonne. Tout ce qu'il lit reste DANS SON contexte à lui, pas dans celui de l'orchestrateur. Il peut lire 40 000 tokens de code sans polluer la session principale.

Étape 5 — Rapport + mort. L'agent forké renvoie un rapport bref (1 000 à 3 000 tokens en général) à l'orchestrateur. Puis il MEURT. Son contexte est jeté. Tout ce qu'il a lu disparaît. Il ne reste que le rapport.

Schéma du cycle de vie

```
T0I: /audit-security
  |
  v
[ORCHESTRATEUR] reconnaît la commande
  |
  v
  FORK une sous-session
  |
  v
[AGENT FORKÉ] né, contexte vide
- lit le SKILL.md (instructions)
- lit 8 fichiers de code (40 000 tokens)
- raisonne, classe, écrit le rapport
  |
  v
renvoie le rapport (1 200 tokens)
  |
  v
[AGENT FORKÉ] meurt, contexte détruit
  |
  v
[ORCHESTRATEUR] reçoit le rapport
(n'a pas vu les 40 000 tokens de code)
```

Pourquoi c'est génial

Cette mécanique sauve la fenêtre de contexte. L'orchestrateur n'a pas à porter le poids de tout ce que ses sous-agents lisent. Il reçoit juste les conclusions.

Reprenons l'exemple. L'agent `audit-security` peut lire 40 000 tokens de code. Il produit un rapport de 1 200 tokens. L'orchestrateur ne voit jamais les 40 000 tokens. Il voit juste le rapport.

Si on n'avait pas le fork, un agent unique aurait dû tout charger dans son contexte. Après quelques skills appelés, sa fenêtre serait pleine. Le système deviendrait lent. Les erreurs s'accumuleraient.

Conséquence pratique : chaque rappel = un agent neuf

C'est la règle d'or. Si tu rappelles `/audit-security` une deuxième fois (par exemple parce que la première a trouvé des problèmes, tu les as corrigés, et tu veux revérifier), tu obtiens un agent NEUF. Pas le même agent. Pas un agent qui se souvient.

Ce nouvel agent ne sait pas qu'il est le numéro 2. Il ne sait pas ce que l'agent précédent a dit. Il refait son travail depuis zéro. La coordination ne vit pas chez les sous-agents. Elle vit chez l'orchestrateur.

C'est comme appeler deux fois SOS-Plombier. La deuxième fois, ils t'envoient peut-être un plombier différent. Le nouveau plombier ne sait pas ce que le premier a fait. Il doit te poser les mêmes questions. Sauf si TU lui racontes l'historique. Ici, c'est l'orchestrateur qui joue le rôle du "TU".

À retenir : *Chaque invocation de skill = un sous-agent neuf, contexte vide, qui exécute sa mission puis meurt. Tout ce que le sous-agent lit reste chez lui. Seul son rapport remonte. Cela protège la fenêtre de contexte de l'orchestrateur.*

Chapitre 7 — Skill atomique vs orchestrateur

Un point qui fait souvent buguer les débutants : un skill *orchestrateur* est-il une chose DIFFÉRENTE d'un skill *atomique* (qui fait une seule tâche) ?

Réponse : **NON**. C'est la même boîte. C'est le même format de fichier. C'est le même mécanisme de fork. La différence est UNIQUEMENT dans le contenu du corps du `SKILL.md`.

Skill atomique

Un skill atomique fait UNE chose. Il a son frontmatter, son corps avec instructions précises, et il s'exécute en autonomie complète.

Exemples : `notify`, `commit-sc`, `audit-security`, `plan-feature`, `simplify`.

Le corps du `SKILL.md` contient des instructions style "fais ceci, vérifie cela, produis ce rapport". Il n'invoque jamais d'autres skills.

Skill orchestrateur

Un skill orchestrateur a EXACTEMENT le même format de fichier. Le frontmatter est similaire. Les `allowed-tools` peuvent être plus larges (souvent `Read`, `Write`, `Edit`, `Bash`, `Grep`, `Glob`).

La différence se trouve dans le CORPS du `SKILL.md`. L'orchestrateur a, dans son corps, des lignes qui disent "Maintenant, lance `/autre-skill`", "Maintenant, lance `/encore-un-autre`". C'est une chaîne de commandes.

Exemples sur ServiceCat : `implement`, `work-issues`, `work-findings`.

Tableau comparatif

	Skill atomique	Skill orchestrateur
Format de fichier	SKILL.md	SKILL.md
Frontmatter	Oui	Oui
Corps avec instructions	Oui	Oui
Appelle d'autres skills ?	Non	Oui (c'est sa raison d'être)
Combien de sous-agents forkés ?	1 (lui-même)	Plusieurs (un par sous-skill appelé)
Exemple	audit-security	implement

La magie est dans le corps, pas dans la nature

C'est important de bien comprendre : un orchestrateur n'est PAS un agent spécial avec des super-pouvoirs. C'est un skill comme les autres. Mais son corps a été écrit pour qu'il "tape" des slash-commands, et donc forke d'autres sous-agents.

Techniquement, l'orchestrateur est juste un agent forké qui sait taper des slash-commands.

Métaphore

Reviens à l'atelier de jouets. Tu as deux types d'artisans.

- L'artisan **atomique** sait peindre. Il prend un jouet, il le peint, il le rend.
- L'artisan **orchestrateur** sait coordonner. Il prend une commande, il envoie le jouet chez le coupeur, puis chez le ponçage, puis chez le peintre, puis chez l'emballleur. Lui-même ne touche pas au jouet. Mais il SAIT taper sur l'épaule des autres pour leur dire "à ton tour".

Les deux ont le même contrat de travail (même format). Mais leur job décrit dans leur contrat est différent.

À retenir : Atomique vs orchestrateur, c'est une différence de mission, pas de nature. Même fichier, même mécanisme. La différence est dans ce que le corps du SKILL.md INSTRUIT de faire.

Chapitre 8 — Décortilage d'un workflow réel

On va maintenant prendre le skill orchestrateur le plus important de ServiceCat, `implement`, et regarder son corps pour comprendre comment il enchaîne ses sous-agents.

Le fichier complet est dans `/Users/moussab/Github/genai-demo/.claude/skills/implement/SKILL.md`.

Le frontmatter

```
---
name: implement
description: Full REPL loop orchestrator. Runs plan → explore → implement → test → review
allowed-tools: Read, Write, Edit, Bash, Grep, Glob
---
```

Note les outils : Read, Write, Edit, Bash, Grep, Glob. C'est plus large que `audit-security`. Pourquoi ? Parce qu'un orchestrateur doit pouvoir lancer des commandes (donc `Bash`), modifier des fichiers (donc `Write` et `Edit`), pas juste lire.

La description annonce déjà la chaîne : plan → explore → implement → test → review → audit → commit → PR. Huit étapes.

Le corps en 4 phases

Le corps de `implement` est organisé en 4 phases. Voyons-les.

Phase 0 — Plan. Le corps dit explicitement "Run `/plan-feature` with the user's request". L'orchestrateur APPELLE le skill atomique `plan-feature`. Ce skill produit un plan avec 6 critères d'acceptation (AC-1 à AC-6).

Si le plan touche plus de 5 fichiers ou 200 lignes, l'orchestrateur pause et demande confirmation à l'humain. C'est un mécanisme de sécurité.

Phase 1 — Per Step. Pour chaque étape du plan, l'orchestrateur enchaîne 7 sous-étapes :

1. `/explore-codebase` (sous-agent A – lit le code existant)
2. `Implement` (sous-agent B – crée le code, peut être `/new-endpoint`, `/new-sc`)
3. `make lint && make test` (commande directe, pas un skill)
4. `Inner Fix Loop` si erreur (3 tentatives max)
5. `/simplify` (sous-agent C – nettoie le code)
6. `/audit-security` (sous-agent D – vérifie la sécurité)
7. `/commit-sc` (sous-agent E – sauvegarde dans Git)

Chacun de ces 7 appels forke un sous-agent neuf. Le contexte de chaque sous-agent reste chez lui.

Phase 2 — Acceptance Gate. L'orchestrateur vérifie que les 6 critères d'acceptation passent. Pas "je pense", VRAIMENT en lançant les tests. Si un critère échoue, retour à la phase 1.

Phase 3 — PR. L'orchestrateur appelle :

1. `/review-pr` (sous-agent – chasse les bugs)
2. `/create-pr` (sous-agent – crée la pull request)
3. `/devops watch` (sous-agent – surveille CI)
4. Si CI rouge, route vers la bonne phase

Arbre des invocations

Voici l'arbre complet des sous-agents qu'un seul appel de `/implement` peut générer pour une feature à 3 étapes :

```
/implement (orchestrateur principal)
├── /plan-feature (sous-agent: génère le plan AC-1..AC-6)
├── ÉTAPE 1/3
│   ├── /explore-codebase (sous-agent A1)
│   ├── /new-endpoint      (sous-agent B1)
│   ├── /simplify          (sous-agent C1)
│   ├── /audit-security    (sous-agent D1)
│   └── /commit-sc         (sous-agent E1)
├── ÉTAPE 2/3
│   ├── /explore-codebase (sous-agent A2)
│   ├── /frontend-design  (sous-agent B2)
│   ├── /simplify          (sous-agent C2)
│   ├── /audit-security    (sous-agent D2)
│   └── /commit-sc         (sous-agent E2)
├── ÉTAPE 3/3
│   ├── /explore-codebase (sous-agent A3)
│   ├── /new-scorecard     (sous-agent B3)
│   ├── /simplify          (sous-agent C3)
│   ├── /audit-security    (sous-agent D3)
│   └── /commit-sc         (sous-agent E3)
├── /review-pr (sous-agent F)
├── /create-pr (sous-agent G)
└── /devops watch (sous-agent H)
```

Pour une feature à 3 étapes, on a forké **17 sous-agents** (1 plan + 5×3 par étape + 3 finaux = 1 + 15 + 3 = 19, en fait — on a un peu arrondi pour la clarté). Chacun avec son contexte propre, sa mission, sa mort à la fin.

Le format de sortie

`implement` doit aussi communiquer son avancement. Le corps du SKILL.md dit :

```
After each step, output one line:  
[F-12 step 3/7] ✅ Implemented POST /scorecards/{id}/versions (commit a1b2c3d)
```

Et à la fin :

```
F-12 – Add scorecard versioning with comparison view  
✅ Plan generated (AC-1..AC-6 set)  
✅ 7 steps implemented  
✅ All ACs pass  
✅ Reviewed: 0 critical, 0 high  
✅ PR opened: <link>  
✅ CI: green  
  
Ready for human review.
```

C'est l'orchestrateur qui consolide. Les sous-agents ne le savent pas. Eux ont fait leur petite mission et sont morts. Le résumé est construit côté orchestrateur, à partir des rapports qu'il a reçus.

À retenir : *Un orchestrateur, c'est un skill dont le corps est un menu d'invocations d'autres skills. Il ne fait pas le travail. Il dirige. Et chaque invocation génère un nouveau sous-agent qui démarre vierge, fait sa mission, meurt.*

Chapitre 9 — Le piège du mega-agent

Maintenant que tu comprends bien le modèle orchestré, voyons l'alternative : tout faire dans UN seul gros agent. Spoiler : c'est très tentant, et c'est presque toujours une mauvaise idée.

Le scénario

Imagine que tu n'utilises pas de skills. Tu ouvres Claude Code, tu tapes :

“Ajoute le versionnement des scorecards à ServiceCat. Fais le plan, lis le code, crée le code backend, crée l'écran frontend, lance les tests, fais une review de sécurité, et fais le commit.”

L'agent va essayer de tout faire dans la même conversation. Le même contexte de session enfle.

Les 7 problèmes du mega-agent

Problème 1 — La facture explose. Tu paies à chaque tour de conversation, et le prix dépend du nombre de tokens dans le contexte. Plus le contexte grossit, plus chaque réponse coûte cher. Sur la feature F-12 de ServiceCat, ça donne **~960 000 tokens** en mega-agent contre **~120 000 tokens** en orchestration. Facteur 8.

Problème 2 — Le drift. Au bout de quelques heures de discussion, l'agent oublie les règles posées au début. Il commence à enfreindre `CLAUDE.md` sans le faire exprès. Plus la conversation est longue, plus il dérive.

Problème 3 — La contamination des phases. Avant de finir de planifier, il commence à coder. Avant de finir de coder, il commence à tester. Tout se mélange dans son contexte. Les conclusions de l'étape précédente influencent (parfois faussement) l'étape suivante.

Problème 4 — Les gates symboliques. Tu lui demandes "fais aussi un audit de sécurité". Il dit "OK, j'ai audité, tout est OK". Mais en fait, dans le même contexte que celui qui a écrit le code, il ne voit pas ses propres erreurs. C'est comme demander à quelqu'un de corriger son propre devoir : il ne voit pas ses fautes.

Problème 5 — Pas de parallélisme. Un mega-agent ne peut pas faire deux choses en même temps. Avec une orchestration, on pourrait forker deux sous-agents simultanés (par exemple un qui audite, un qui écrit la doc).

Problème 6 — Aucun garde-fou outils. Le mega-agent a accès à TOUS les outils. Il peut tout lire, tout écrire, tout supprimer. Une erreur peut faire des dégâts. Avec les skills, chaque sous-agent n'a que les outils dont il a besoin.

Problème 7 — Débuggage cauchemardesque. Si quelque chose va mal, tu dois relire des heures de conversation pour trouver où ça a dérapé. Avec l'orchestration, chaque sous-agent a un rapport court. Tu sais exactement quel rapport est en cause.

Tableau comparatif chiffré

Pour la feature F-12 (versionnement des scorecards) sur ServiceCat :

Critère	Mega-agent	Orchestration
Tokens totaux consommés	~960 000	~120 000
Coût relatif	x8	x1
Nombre de sous-agents	0 (un seul gros)	~19
Contexte de l'orchestrateur en fin	Plein, ~750 000 tokens	Léger, ~25 000 tokens
Fiabilité du résultat	Variable	Élevée (gates intégrés)
Audit de sécurité séparé	Non (auto-audit douteux)	Oui (sous-agent neuf)
Reprise si erreur	Repartir de zéro	Reprendre à l'étape concernée

Métaphore : le chef qui cuisine seul pour 50

Imagine un restaurant qui doit servir 50 couverts un samedi soir. Tu as deux options.

Option mega-agent : UN SEUL chef qui fait tout. Il coupe les légumes, sauce la viande, cuit les pâtes, dresse les assiettes, fait le service, encaisse. Il est très bon partout. Mais il est seul. Au bout de 2h, il est cramé. Il fait des erreurs. Il oublie une commande. Il sert un plat froid.

Option orchestration : UNE BRIGADE. Un chef qui coordonne. Un commis aux légumes. Un saucier. Un cuisinier des pâtes. Un dresseur. Un serveur. Une caissière. Chacun fait UNE chose, à fond. Le chef regarde le service, donne les signaux, vérifie chaque assiette avant la sortie.

La brigade gagne. Tout le temps. Pour les mêmes raisons : spécialisation, parallélisme, points de contrôle, fatigue répartie.

À retenir : Un seul gros agent qui fait tout = un chef seul pour 50 couverts. C'est tentant, c'est plus simple à mettre en place, mais c'est plus cher, plus lent, moins fiable. L'orchestration est l'analogue d'une brigade de cuisine professionnelle.

Chapitre 10 — Comment l’orchestrateur décide de l’ordre et quand rappeler un skill

Tu comprends que l’orchestrateur appelle des skills. Mais COMMENT décide-t-il dans quel ordre ? Et QUAND décide-t-il de rappeler un skill une deuxième fois ?

Les 3 sources de l’ordre

L’ordre d’enchaînement des skills vient de TROIS sources qui se combinent.

Source 1 — La section `## Process` figée du SKILL.md. Quand tu écris un orchestrateur, tu fixes l’ordre dans son corps. Par exemple, le corps de `implement` dit littéralement : “Phase 0 : `/plan-feature`. Phase 1 : pour chaque étape, `/explore-codebase` puis `Implement` puis `tests` puis `/simplify` puis `/audit-security` puis `/commit-sc`.” Cet ordre est écrit dans le marbre du fichier. Il ne change pas.

Source 2 — Le plan dynamique de `/plan-feature`. Quand l’orchestrateur appelle `plan-feature`, il reçoit en retour une liste d’étapes spécifiques à la feature demandée. Par exemple, pour F-12 : “étape 1 : créer la migration BD ; étape 2 : ajouter l’endpoint ; étape 3 : ajouter l’écran”. La liste a 3 entrées cette fois, peut-être 7 la fois suivante. L’orchestrateur boucle dessus.

Source 3 — Les branches déclenchées par les gates rouges. Si une gate échoue, l’orchestrateur prend un détour. Par exemple : tests rouges → retour étape 4 (Inner Fix Loop), audit rouge → retour étape 2 (re-coder), CI rouge → routage automatique vers la phase qui correspond au type d’erreur.

L’ordre final est donc : SOURCE 1 (le squelette figé) appliqué SUR SOURCE 2 (les étapes du plan), avec SOURCE 3 (les boucles correctives) qui s’active quand quelque chose casse.

Les 5 scénarios de rappel d’un skill

Pendant un seul `/implement`, le même skill peut être appelé plusieurs fois. Voici les 5 scénarios.

Scénario 1 — Boucle “for each step”. L’orchestrateur fait une étape à la fois. À chaque étape, il appelle `/explore-codebase`. Si le plan a 5 étapes, il appelle `/explore-codebase` 5 fois. À chaque appel, un sous-agent neuf.

Scénario 2 — Inner Fix Loop. Si les tests rouges après une implémentation, l’orchestrateur essaie de corriger. Première tentative : il analyse l’erreur, propose une correction, relance les

tests. Si rouge encore, deuxième tentative. Troisième tentative. Maximum 3. Au-delà, il s'arrête et appelle l'humain.

Scénario 3 — Audit rouge → re-audit. `/audit-security` trouve des problèmes. L'orchestrateur les corrige. Il rappelle `/audit-security`. Mais c'est un sous-agent NEUF. Il refait tout l'audit depuis zéro. Il ne sait pas ce qu'il a dit la fois précédente.

Scénario 4 — CI rouge → routage. La PR est créée, mais la CI échoue. L'orchestrateur lit le log de CI. Si l'erreur est un lint, il appelle un skill qui corrige le lint. Si c'est un test, il appelle un sous-agent qui corrige le test. Si c'est un audit, idem. Routage selon le type d'erreur.

Scénario 5 — Auto-triage. Pendant l'exploration, l'agent découvre un bug qui n'était pas dans la mission. L'orchestrateur appelle `/triage` pour créer un ticket, puis CONTINUE sa mission principale. Le bug est noté pour plus tard.

Chaque rappel = un agent neuf

C'est la règle d'or qu'on a déjà vue, mais qui mérite d'être martelée : à chaque rappel, un sous-agent NEUF est forké. Il démarre avec un contexte vierge. Il ne sait pas qu'il est le numéro 2, 3, ou 17.

Toute la mémoire de coordination vit dans l'orchestrateur, pas chez les sous-agents. C'est l'orchestrateur qui se souvient de "j'ai déjà appelé `/audit-security` trois fois pour cette feature". Les sous-agents, eux, sont des FONCTIONS PURES : on leur donne une entrée, ils produisent une sortie, ils meurent.

À retenir : L'ordre vient du SKILL.md figé + du plan dynamique + des branches de correction. Les rappels sont fréquents (boucle d'étapes, fix loop, re-audit, routage CI). Mais chaque rappel = un agent neuf. La mémoire de coordination vit dans l'orchestrateur seul.

Chapitre 11 — Où vit la mémoire

Voici un chapitre essentiel souvent mal compris : OÙ exactement vit l'information dans un système d'orchestration ?

Il y a TROIS lieux distincts. Confondre ces trois lieux, c'est ne rien comprendre. Distinguer les trois, c'est tout comprendre.

Lieu 1 — Le contexte de session (la RAM)

Le contexte de session, c'est la *RAM* (mémoire vive) de l'agent IA. C'est ce qu'il "voit" en ce moment. C'est sa fenêtre de contexte. C'est tout ce qu'il peut traiter en une réponse.

Caractéristiques : - Temporaire. Jeté à la fin de la session. - Limité (1 million de tokens pour Claude Opus 4.7 1M, ou ~750 000 mots français). - Modifiable à chaque tour.

Quand un sous-agent meurt, sa RAM est libérée. Tout ce qu'il avait dedans disparaît, sauf ce qu'il a renvoyé en rapport à l'orchestrateur.

Lieu 2 — Le disque du repo (permanent)

Tout ce qui est écrit dans des fichiers du projet (`.py` , `.md` , `.json`) ou dans la base de données est *persistant*. Cela survit à la fin de la session.

Caractéristiques : - Permanent. - Lisible par tout le monde (humains, agents futurs). - Versionné par Git.

Sur ServiceCat, cela inclut : tout le code, les migrations de BD, les `SKILL.md` , le `CLAUDE.md` , les findings écrits en base, les audit logs, les commits Git.

Lieu 3 — L'auto-memory inter-sessions

Claude maintient une petite mémoire personnelle qui survit entre les sessions. Sur ServiceCat, elle est dans `~/.claude/projects/servicecat/memory/` .

Caractéristiques : - Très petite (quelques kilo-octets). - Stocke des préférences, des erreurs déjà commises, des corrections de l'équipe. - N'est PAS le code du projet. C'est l'expérience de Claude SUR le projet.

Exemple : si Claude a commis une erreur (par exemple : "j'ai oublié d'utiliser le helper `httpx.AsyncClient` "), et que l'humain l'a corrigé, l'auto-memory note ça. À la prochaine session, Claude n'oubliera plus.

Tableau récapitulatif

Lieu	Permanent ?	Taille	Visible où ?
Contexte de session (RAM)	Non	Jusqu'à 1M tokens	Dans le tour de conversation en cours
Disque du repo	Oui	Plusieurs Go	Fichiers, Git, BD
Auto-memory	Oui	Quelques Ko	Dossier ~/.claude/projects/.../memory/

Comment le forking sauve la fenêtre de contexte

Reviens à l'exemple. L'agent `audit-security` est forké. Il lit 40 000 tokens de code. Tout ça vit dans SA RAM, pas dans celle de l'orchestrateur.

Quand l'agent meurt, sa RAM disparaît. Mais il a écrit un rapport de 1 200 tokens. Ces 1 200 tokens, eux, entrent dans la RAM de l'orchestrateur.

Bilan : - L'orchestrateur "voit" 1 200 tokens (le rapport). - Il ne paye pas pour les 40 000 tokens lus par le sous-agent (sauf pour la facture du sous-agent lui-même, qui est isolée).

Sans forking, l'orchestrateur aurait DÛ lire les 40 000 tokens lui-même, et ils resteraient dans sa RAM jusqu'à la fin de la session. Au bout de 3 ou 4 skills, sa RAM serait saturée.

Les chiffres concrets pour Claude Opus 4.7 1M

Élément	Valeur
Fenêtre de contexte totale	1 000 000 tokens
Coût en tokens d'une feature complète ServiceCat	~37 000 tokens dans la RAM de l'orchestrateur
Coût relatif (% de la fenêtre)	~3,7%
Combien de features tient-on dans une session ?	~25 avant débordement

Tu vois ? On a énormément de marge. Le système est conçu pour rester très loin de la limite.

Les 3 mécanismes anti-débordement

Claude Code combine trois mécanismes pour ne JAMAIS déborder.

Mécanisme 1 — Forking systématique. Tout gros travail de lecture est délégué à un sous-agent. Les volumes ne remontent pas à l'orchestrateur.

Mécanisme 2 — Compaction automatique. Quand la conversation principale devient longue, Claude compacte les vieux messages : il les résume. "Tu m'as posé 15 questions sur la BD, j'ai répondu, voici le résumé : on a clarifié la structure des tables X, Y, Z." Les 15 tours deviennent 1 tour de résumé.

Mécanisme 3 — Persistance externe. Au lieu de garder une info dans le contexte, on l'écrit dans un fichier ou dans la BD. La prochaine fois qu'on en a besoin, on relit le fichier. Le contexte reste propre.

Les 4 cas pathologiques qui font déborder quand même

Même avec ces protections, on peut déborder. Voici les 4 cas problématiques.

Cas 1 — Tout-en-un sans fork. L'humain donne une mission énorme à l'agent SANS utiliser de skills. L'agent doit tout charger dans son seul contexte. Débordement assuré.

Cas 2 — Logs verbeux non filtrés. L'agent lance une commande qui produit 500 000 lignes de log et il les avale toutes. Le contexte se remplit en une commande.

Cas 3 — Relectures inutiles de gros fichiers. L'agent relit 10 fois le même fichier de 5 000 lignes. À chaque relecture, ça consomme du contexte.

Cas 4 — Sessions ouvertes sur plusieurs jours. L'humain garde la même session ouverte pendant 3 jours, avec des centaines de messages. La compaction aide mais ne suffit plus. Mieux vaut tuer la session et en démarrer une neuve, comme le dit la slide 12 du talk : *kill the session, keep the skill*.

À retenir : Trois lieux de mémoire. La RAM est temporaire et limitée. Le disque du repo est permanent. L'auto-memory survit entre sessions. Le forking sauve la fenêtre. Le but, c'est de finir une feature avec une RAM légère.

Chapitre 12 — Les gates en détail

On a parlé de *gates* (barrières, points de contrôle) plusieurs fois. C’est temps de les comprendre à fond. C’est probablement le concept le plus important de tout ce guide. La slide 13 du talk en parle.

Définition simple

Une *gate* (le mot anglais signifie “porte” ou “barrière”) est un point de passage obligatoire dans un workflow. Soit on passe, soit on est renvoyé en arrière. Pas de demi-mesure.

Trois analogies pour bien comprendre

Analogie 1 — La douane à l’aéroport. Tu arrives avec ton sac. Tu passes le scanner. Si tout est OK, on te laisse passer. Si on détecte un objet interdit, on t’arrête. Tu ne peux pas dire “mais s’il vous plaît, faites une exception”. La gate est binaire : OUI ou NON.

Analogie 2 — Le contrôle technique de la voiture. Tu arrives au garage. Le contrôleur vérifie 50 points. Freins, phares, pneus, échappement. À la fin, soit ta voiture est certifiée pour rouler, soit elle ne l’est pas. Si elle a un défaut majeur, tu DOIS réparer et REPASSER l’inspection. On ne te donne pas le certificat “en attendant”.

Analogie 3 — L’examen final à l’école. Tu as suivi un cours toute l’année. À la fin, il y a un examen. Soit tu as la moyenne et tu passes en année supérieure, soit tu redoubles. L’enseignant peut être sympa, mais les règles sont les règles.

Gate vs Test

Beaucoup de débutants confondent “gate” et “test”. Voyons la différence.

	Test	Gate
Quoi ?	Une vérification de comportement	Une décision de passage
Résultat	Passe ou échoue	Continue ou retour
Conséquence d’un échec	Tu apprends qu’il y a un problème	Tu es bloqué tant que ce n’est pas corrigé
Exemple	“Le bouton fait bien telle action”	“Pas plus de 0 critiques sécurité”

Une gate utilise souvent des tests pour décider. Mais une gate n'est pas un test : c'est une décision de TRANSITION entre deux phases du workflow.

Les 6 AC de ServiceCat

ServiceCat a 6 critères d'acceptation (Acceptance Criteria), notés AC-1 à AC-6. Ces 6 critères ENSEMBLE forment la gate finale d'une feature.

Critère	Catégorie	Ce qui doit passer
AC-1	Fonctionnel	Tous les comportements décrits marchent — happy path + au moins 2 cas limites
AC-2	Tests	Tests unitaires + tests d'intégration + tests de sécurité passent avec $\geq 80\%$ de couverture
AC-3	Sécurité	<code>/audit-security</code> clean — les 5 gardes présents, S6-S8 OK
AC-4	Qualité	<code>/simplify</code> clean — pas de duplication, pas de patterns enfreints
AC-5	Lint & Build	<code>make lint</code> et <code>pnpm build</code> passent sans erreur
AC-6	Frontend	Types générés, hooks via TanStack Query, i18n, dark mode, <code>/frontend-design</code> utilisé

Si UN seul des 6 échoue, la feature N'EST PAS FINIE. L'orchestrateur retourne à la phase concernée et corrige.

Exemple raconté pas à pas

Imaginons que tu as demandé à `/implement` de “ajouter le bouton Partager sur les scorecards”. Voici ce qui se passe :

1. Plan généré. AC-1 à AC-6 définis.
2. Étape 1 : créer l'endpoint backend `POST /scorecards/{id}/share` . Code créé.
3. `/audit-security` lancé. Il trouve que la rate-limit manque (S4). **Gate AC-3 : ROUGE.**
4. L'orchestrateur retourne au code. Il ajoute `Depends(rate_limit(...))` .
5. `/audit-security` relancé. Cette fois CLEAN. **Gate AC-3 : VERT.**
6. Étape 2 : créer l'écran. Code créé.
7. `make test` lancé. Un test échoue. **Gate AC-2 : ROUGE.**

8. Inner Fix Loop. Tentative 1 : analyser l'erreur, corriger. Tests relancés. VERT. **Gate AC-2 : VERT.**
9. `pnpm lint` lancé. Une erreur de lint. **Gate AC-5 : ROUGE.**
10. Corriger. Relancer. **Gate AC-5 : VERT.**
11. Tous les AC sont VERT.
12. `/create-pr` crée la pull request.

Sans les gates, le PR aurait été créé à l'étape 2 avec un trou de sécurité. Avec les gates, on retourne en arrière à chaque problème.

Ce que les gates NE SONT PAS

Les gates ne sont PAS des suggestions. Elles ne sont PAS des conseils gentils. Elles ne sont PAS contournables.

- Une gate n'est pas un avertissement qu'on peut ignorer.
- Une gate n'accepte pas le compromis "je sais que c'est un peu cassé, mais on verra plus tard".
- Une gate ne tolère pas "ça passera bien comme ça".

C'est tout l'intérêt. Sans la rigidité des gates, le système se dégrade. Les gates sont le squelette qui empêche le projet de s'effondrer.

La règle des 3 (slide 13)

Moussa rappelle trois règles dans son talk :

Règle 1 — Gate : chaque transition entre phases est une gate. Pas de "et si on passait directement à la PR sans audit ?". Non.

Règle 2 — Stopping : après 3 échecs consécutifs dans la même boucle, on s'arrête. On appelle l'humain. On ne tourne pas en rond.

Règle 3 — Human sync points : certaines décisions ne sont JAMAIS automatisées. Le merge en main. La résolution d'un conflit de design. Le go-no-go d'une feature majeure. Toujours un humain.

À retenir : Une gate est un point de passage binaire. OUI ou NON, jamais "peut-être". Les 6 AC de ServiceCat forment ensemble la gate finale d'une feature. Un seul AC rouge = feature pas finie. Sans gates, pas d'ingénierie : juste du bricolage.

Chapitre 13 — Les 8 règles d’or pour orchestrer un workflow multi-agents

Si tu retiens ces 8 règles, tu sais l’essentiel pour démarrer toi-même un système orchestré.

Règle 1 — Un skill = une mission

Chaque skill doit pouvoir s’expliquer en une phrase courte. Si tu dois écrire un paragraphe pour décrire ce qu’il fait, c’est qu’il en fait trop. Découpe-le.

Règle 2 — Frontmatter clair, corps détaillé

Le frontmatter doit être court (3 lignes : name, description, allowed-tools). Le corps peut être long (instructions précises, format de sortie, interdits). C’est la séparation entre “qui suis-je” et “comment je travaille”.

Règle 3 — Limite les outils

Donne à chaque skill UNIQUEMENT les outils dont il a besoin. `audit-security` n’a que Read, Grep, Glob (lecture seule). C’est un garde-fou. Un sous-agent qui n’a pas le droit d’écrire ne peut pas faire de dégâts.

Règle 4 — Tout passage entre phases est une gate

Pas de “j’ai fini de coder, je passe directement à la PR”. Entre chaque phase : une vérification. Tests verts, audit clean, lint OK. Si rouge : retour en arrière.

Règle 5 — Inner Fix Loop limité à 3 tentatives

Si une correction échoue 3 fois, ce n’est plus une question de patch local. C’est probablement un problème de conception. Stop. Appelle l’humain.

Règle 6 — La mémoire de coordination vit dans l’orchestrateur

Les sous-agents sont des fonctions pures. Ils ne se souviennent pas. L’orchestrateur seul tient le fil. Si l’orchestrateur perd son contexte, le travail est foutu. Donc protège son contexte (= utilise le forking massivement).

Règle 7 — Persiste l'important hors session

Tout ce qui doit survivre à la fin de la session DOIT être écrit quelque part : fichier, BD, commit Git, audit log. Le contexte de session est volatile. Le disque est permanent.

Règle 8 — Jamais de merge sans humain

Aucune automatisation ne doit décider du passage en production. La PR est créée par l'orchestrateur. Le merge est fait par un humain. C'est la dernière barrière. Slide 13 du talk : *human sync points*.

À retenir : Une mission par skill. Outils limités. Gates obligatoires. Fix loop à 3 tentatives max. Mémoire dans l'orchestrateur. Persistance hors session. Humain pour les décisions critiques. Voilà le squelette d'un système d'orchestration sain.

Chapitre 14 — Démo ServiceCat décodée

Le talk de Moussa contient 3 moments de démo (slides 14 à 18). On va les décoder en langage simple.

Moment 1 — Du backlog à la PR (slide 15)

Ce que tu vois : Moussa tape `/work-issues` dans Claude Code. Quelques minutes plus tard, une pull request apparaît sur GitHub, prête à reviewer.

Ce qui se passe vraiment sous le capot :

1. `/work-issues` est un orchestrateur. Il commence par lister les issues GitHub ouvertes du projet.
2. Il sélectionne celle avec la priorité la plus haute ($P0 > P1 > P2 > P3$).
3. Il transforme l'issue en feature request et lance `/implement` dessus.
4. `/implement` exécute le cycle complet : plan, explore, code, test, simplify, audit, commit.
5. À la fin, `/create-pr` crée la PR. `/devops watch` surveille CI.
6. Si CI échoue, routage automatique. Sinon, PR prête.

Combien de sous-agents forkés ? Entre 15 et 30 selon la taille de l'issue.

Moment 2 — De l'audit aux corrections (slides 16-17)

Ce que tu vois : Moussa tape `/audit-service payment-svc`. Quelques minutes plus tard, une liste de findings (résultats de l'audit) apparaît dans l'interface ServiceCat. Puis il tape `/work-findings`, et des PR commencent à s'ouvrir SUR LES AUTRES REPOS pour corriger les problèmes.

Ce qui se passe vraiment sous le capot :

1. `/audit-service` clone le repo du service `payment-svc`.
2. Il fait tourner les 4 scorecards (Security, Observability, Documentation, Reliability) dessus.
3. Chaque scorecard produit des *findings* (problèmes trouvés) avec un champ `auto_fixable: true/false`.
4. Les findings sont enregistrés en BD avec leur sévérité (CRITICAL, HIGH, MEDIUM, LOW).
5. `/work-findings` lit la BD, sélectionne les findings auto-fixables, et pour chacun : - Clone le repo source. - Crée une branche. - Applique le fix suggéré dans le finding. - Ouvre une PR sur le repo source.

Important : ServiceCat ne modifie jamais directement les repos des services. Il propose toujours via PR. L'équipe propriétaire du service décide.

Moment 3 — De la notification à la résolution (slide 18)

Ce que tu vois : Une notification Slack arrive à l'équipe `team-payments` : "3 nouvelles findings de sévérité HIGH sur payment-svc". Un membre clique sur le lien. ServiceCat ouvre la page du finding avec le fichier en cause, la ligne, la remédiation. Le membre approuve la PR proposée.

Ce qui se passe vraiment sous le capot :

1. Quand un finding HIGH ou CRITICAL est créé, le système trouve l'équipe propriétaire via `Service.owner_team_id`.
2. Le skill `/notify` est invoqué. Il envoie le message Slack au canal de l'équipe.
3. Le message inclut un lien direct vers la finding dans ServiceCat.
4. L'humain clique, voit le contexte, et approuve ou rejette.

Pourquoi c'est fort : la chaîne entière (audit → finding → notification → PR → review humaine) est faite par des agents. L'humain n'intervient qu'à la fin, pour approuver. C'est l'incarnation parfaite du *human sync point*.

À retenir : Les 3 démos montrent une chaîne complète. Du backlog (issues à faire) au code livré (PR mergée), tout est orchestré. L'humain intervient à des points choisis (validation, approbation), jamais pour faire le travail répétitif.

Chapitre 15 — Démarrer lundi matin

La slide 19 du talk s'appelle *Start Monday*. On va te dire concrètement comment commencer, dès lundi matin, à mettre en place de l'orchestration sur TON projet.

Étape 1 — Installe Claude Code

Si ce n'est pas déjà fait, installe Claude Code via le site officiel. C'est l'environnement où tu vas faire vivre tes agents.

Une fois installé, ouvre ton premier projet. Tu vas voir qu'il fonctionne déjà sans skills, juste comme un chat.

Exemple concret : ouvre Claude Code dans le dossier de ton projet. Tape "explique-moi ce que fait ce code" en pointant un fichier. Tu verras Claude répondre. C'est la base.

Étape 2 — Crée ton `CLAUDE.md`

Crée un fichier `CLAUDE.md` à la racine de ton projet. Ce sera ton livre de règles de la maison. Il doit contenir au moins :

- Tech stack (langages, frameworks, BD).
- Conventions de nommage (snake_case, camelCase, etc.).
- Règles de sécurité (les 5 gardes ServiceCat ou les tiennes).
- Critères d'acceptation (AC-1 à AC-6 ou les tiens).
- Repository layout (la structure des dossiers).

Exemple concret : ouvre `/Users/moussab/Github/genai-demo/CLAUDE.md` pour voir le `CLAUDE.md` de ServiceCat. C'est un excellent modèle. Adapte-le à ton projet.

Étape 3 — Écris ton premier skill atomique

Choisis UN problème répétitif que tu rencontres souvent. Par exemple : “rédiger des messages de commit clairs”. Crée un dossier `.claude/skills/commit-clear/` avec un fichier `SKILL.md` dedans.

Mets-y un frontmatter :

```
---
name: commit-clear
description: Generates a clean conventional commit message based on staged changes.
allowed-tools: Bash, Read
---
```

Et un corps avec les instructions pour générer le message. Teste avec `/commit-clear`.

Exemple concret : regarde `/Users/moussab/Github/genai-demo/.claude/skills/commit-sc/SKILL.md` comme inspiration. Adapte au format de ton équipe.

Étape 4 — Compose un orchestrateur quand le besoin viendra

Quand tu as 5-6 skills atomiques qui marchent bien et que tu te retrouves à les enchaîner manuellement à chaque feature, c’est le signe : il est temps de créer un orchestrateur.

Crée un skill orchestrateur (par exemple `/feature-flow`). Dans son corps, écris littéralement la séquence :

```
1. Run /plan-feature
2. For each step:
  - Run /explore-codebase
  - Implement
  - Run /audit-clean
  - Run /commit-clear
3. Run /create-pr
```

Exemple concret : ouvre `/Users/moussab/Github/genai-demo/.claude/skills/implement/SKILL.md` comme modèle d’orchestrateur. Adapte selon ton workflow.

À retenir : Tu n’as pas besoin de tout faire en un jour. Démarre par Claude Code. Ajoute un `CLAUDE.md`. Crée un premier skill atomique. Quand tu en as plusieurs, compose un orchestrateur. C’est un parcours en quelques semaines, pas en un soir.

Glossaire enrichi

Voici les 30 termes clés du tutoriel, définis de façon claire.

Agent IA : Un programme d'intelligence artificielle qui agit (lit, écrit, lance des commandes), pas seulement qui répond à des questions.

Agent forké : Un sous-agent créé temporairement à partir d'un skill, avec un contexte vide. Il vit le temps de sa mission puis meurt.

Atomique (skill atomique) : Un skill qui fait une seule chose précise et qui n'invoque pas d'autres skills.

Auto-memory : La mémoire personnelle de Claude sur un projet, stockée dans `~/.claude/projects/.../memory/`. Survit entre sessions. Très petite.

CLAUDE.md : Le fichier racine d'un projet Claude Code. Auto-chargé au démarrage. Contient les règles globales.

Compaction : Mécanisme qui résume automatiquement les vieux messages d'une longue conversation pour libérer du contexte.

Contexte (de session) : Ce que l'agent IA "voit" en même temps : ta question, l'historique, les fichiers récents. C'est sa mémoire de travail immédiate.

Drift : Phénomène où un agent perd progressivement de vue les règles initiales au fil d'une longue conversation.

Fenêtre de contexte : La taille maximale de tokens qu'un modèle d'IA peut traiter à la fois. 1 million pour Claude Opus 4.7 1M.

Fork : Création d'une copie indépendante d'un agent, avec un contexte vierge, pour exécuter une mission spécifique.

Frontmatter : Les premières lignes (entre deux `---`) d'un fichier SKILL.md, qui définissent name, description, et allowed-tools.

Gate : Point de passage obligatoire dans un workflow. Binaire : OUI ou NON. Si NON, retour à l'étape précédente.

IA (Intelligence Artificielle) : Programme qui imite certains comportements humains : lire, écrire, raisonner, coder.

Inner Fix Loop : Boucle de correction automatique limitée à 3 tentatives. Au-delà, l'orchestrateur arrête et appelle l'humain.

Mega-agent : Un seul agent qui essaie de tout faire dans une seule conversation. Anti-pattern à éviter.

Mémoire de coordination : La mémoire de l'avancement d'une tâche multi-étapes. Vit chez l'orchestrateur, pas chez les sous-agents.

Modèle : La version précise d'une IA (Claude Opus 4.7, Claude Sonnet 4, etc.). Définit sa taille, sa vitesse, sa fenêtre de contexte.

Orchestrateur (skill orchestrateur) : Un skill qui invoque d'autres skills. Même format de fichier qu'un skill atomique, mais son corps contient des appels à des slash-commands.

Persistance externe : Stocker une information hors du contexte de session (fichier, BD, commit Git) pour qu'elle survive.

Prompt : Le message que tu envoies à l'IA. Ta question, ta demande, ton instruction.

RAM : Métaphore pour le contexte de session. La mémoire vive, temporaire, jetée à la fin.

Routage (CI) : Mécanisme par lequel l'orchestrateur, voyant une CI rouge, dirige le fix vers la phase appropriée (lint, test, audit).

Session : Une exécution de Claude Code, du lancement à la fermeture. Contexte de session = ce qui vit pendant cette exécution.

Skill : Une compétence figée dans un dossier `.claude/skills/<nom>/`, avec un fichier `SKILL.md` qui contient ses instructions.

SKILL.md : Le fichier qui décrit un skill. Composé d'un frontmatter (auto-chargé) et d'un corps (lu à l'invocation).

Slash-command : Une commande tapée avec un `/` devant (ex: `/audit-security`). Elle déclenche l'invocation d'un skill.

Sous-agent : Un agent forké, instance temporaire créée pour exécuter un skill puis mourir.

Token : Unité de mesure de texte pour les IA. Environ trois quarts de mot français. Une phrase courte = ~10 tokens.

Vibe coding : Coder à l'instinct avec une IA, sans cadre. Rapide à démarrer, fragile à maintenir.

Vibe engineering : Construire avec des IA dans un cadre rigoureux : skills, gates, AC. Plus structuré, beaucoup plus fiable.

Annexe — Pour aller plus loin

Si tu veux approfondir, voici les fichiers à explorer dans le dépôt ServiceCat :

- `/Users/moussab/Github/genai-demo/CLAUDE.md` — Le livre de règles complet du projet. Modèle à étudier pour ton propre `CLAUDE.md`. Auto-chargé au démarrage de toute session Claude Code dans ce projet.
- `/Users/moussab/Github/genai-demo/.claude/skills/` — Le dossier qui contient les 18 skills. Chaque sous-dossier contient un `SKILL.md`. Va voir `audit-security/SKILL.md` pour un excellent exemple de skill atomique, et `implement/SKILL.md` pour un excellent exemple de skill orchestrateur.
- `/Users/moussab/Github/genai-demo/.claude/settings.json` — Les réglages de session pour ce projet. Auto-chargé au démarrage. Définit les permissions, les hooks, les variables d'environnement.
- **WORKFLOW.md (ou son équivalent dans le projet)** — La description du processus humain de l'équipe : qui fait quoi, quand utiliser quel skill, comment se passer le travail entre humains et agents. Nom libre, lu à la demande.
- **Le talk de Moussa du 18 juin 2026** — Les slides 1 à 19. Chaque slide est explicitement référencée dans ce guide. Une relecture après ce tutoriel devrait être fluide.
- **Pour aller au cœur** — Tu peux ouvrir n'importe quel skill et lire son `SKILL.md`. C'est le matériau pédagogique le plus dense. Chaque `SKILL.md` est une mini-leçon sur comment résoudre UN problème précis avec une IA.

Bon courage. Tu as toutes les clés maintenant. La suite, c'est l'expérience.